Tema 4.1. Composición

1. En C, podemos crear estructuras mayores **componiendo** unas con otras, que suelen describirse como "A tiene-un/tiene-varios B". Pon un ejemplo, empleando `struct`, de una línea de puntos, donde puntos tienen dos coordenadas (`x` e `y`), y la línea esta hecha de dos puntos. Incluye una función para calcular la distancia entre puntos y otra para hallar la longitud de una línea.

Respuesta

En C, la composición se logra utilizando estructuras (`struct`). Por ejemplo, para representar una línea formada por dos puntos, se pueden definir las estructuras `Punto` y `Linea`. Cada `Punto` tiene dos coordenadas (`x` e `y`), y una `Linea` está compuesta por dos puntos. Además, se pueden implementar funciones para calcular la distancia entre dos puntos y la longitud de una línea:

```c
#include <math.h>
#include <stdio.h>

typedef struct {
    float x;
    float y;
} Punto;

typedef struct {
    Punto inicio;
    Punto fin;
} Linea;

float calcularDistancia(Punto a, Punto b) {
    return sqrt(pow(b.x - a.x, 2) + pow(b.y - a.y, 2));
}

float calcularLongitud(Linea linea) {
    return calcularDistancia(linea.inicio, linea.fin);
}

int main() {
    Punto p1 = {0, 0};
    Punto p2 = {3, 4};
    Linea linea = {p1, p2};

    printf("Distancia entre puntos: %.2f\n", calcularDistancia(p1, p2));
    printf("Longitud de la línea: %.2f\n", calcularLongitud(linea));

    return 0;
}
```

Este ejemplo demuestra cómo se pueden combinar estructuras para modelar relaciones entre entidades y cómo se pueden implementar funciones para operar sobre ellas.

2. Ahora transforma ese ejemplo a orientación a objetos con Java, para tener un primer ejemplo de **composición** en orientación a objetos. Crea una

clase `Punto`, y una clase `Linea`. La clase `Punto` debe tener un método para calcular distancia a otro `Punto` y `Linea` debe tener un método para calcular su longitud. Gracias a la ocultación de información, supera a C, garantizando que los puntos sean inmutables, al igual que la línea, que una vez creada, no queremos que se modifique de qué a qué puntos va dicha línea.

Respuesta

En Java, la composición se implementa creando clases que contienen referencias a otras clases. En este caso, se pueden definir las clases `Punto` y `Linea`. La clase `Punto` incluye un método para calcular la distancia a otro punto, y la clase `Linea` utiliza dos objetos `Punto` para representar sus extremos. Además, se garantiza la inmutabilidad mediante el uso de atributos `final` y la ausencia de métodos que modifiquen el estado interno:

```java
public final class Punto {
    private final double x;
    private final double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double calcularDistancia(Punto otro) {
        return Math.sqrt(Math.pow(otro.x - this.x, 2) + Math.pow(otro.y - this.y, 2));
    }
}

public final class Linea {
    private final Punto inicio;
    private final Punto fin;

    public Linea(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public double calcularLongitud() {
        return inicio.calcularDistancia(fin);
    }
}

public class Main {
    public static void main(String[] args) {
        Punto p1 = new Punto(0, 0);
        Punto p2 = new Punto(3, 4);
        Linea linea = new Linea(p1, p2);

        System.out.println("Longitud de la línea: " + linea.calcularLongitud());
    }
}
```

Este enfoque garantiza que los objetos `Punto` y `Linea` sean inmutables, lo que mejora la seguridad y la claridad del código.

3. ¿Qué significa la **multiplicidad** en la composición? En el ejemplo anterior, ¿cuál es la multiplicidad entre `Linea` y `Punto`? Indícalo expresando la multiplicidad en ambas direcciones, de `Linea` a `Punto` y de `Punto` a `Linea`.

Respuesta

La multiplicidad en la composición describe la cantidad de instancias de una clase que están asociadas con una instancia de otra clase. En el ejemplo anterior, la multiplicidad entre `Linea` y `Punto` es de "1 a 2" en la dirección de `Linea` a `Punto`, ya que una línea está compuesta exactamente por dos puntos. En la dirección opuesta, de `Punto` a `Linea`, la multiplicidad es "0 o más", ya que un punto puede no pertenecer a ninguna línea o puede ser parte de múltiples líneas.

En términos de UML, esto se representaría con una línea que conecta las clases `Linea` y `Punto`, con "1" cerca de `Linea` y "2" cerca de `Punto` en una dirección, y "0..*" cerca de `Punto` en la dirección opuesta. Este concepto es fundamental para modelar relaciones entre clases y garantizar que las restricciones de diseño se respeten.

4. ¿Qué significa composición **fuerte** y composición **débil**? ¿Qué consecuencia implica en relación al ciclo de vida de los objetos? Indica a cuál solemos referirnos como **"asociación o agregación"** y a cuál como **"composición"** propiamente.

Respuesta

La composición fuerte y débil se diferencian principalmente en el ciclo de vida de los objetos. En una composición fuerte, los objetos compuestos (contenidos) no pueden existir independientemente del objeto contenedor. Por ejemplo, si una línea contiene puntos y la línea se elimina, los puntos también deberían eliminarse. Esto se conoce como "composición" propiamente dicha.

En cambio, en una composición débil, los objetos contenidos pueden existir independientemente del contenedor. Por ejemplo, si una clase `Grupo` contiene referencias a objetos `Persona`, estas personas

pueden seguir existiendo aunque el grupo se elimine. Esto se conoce como "asociación" o "agregación".

La principal consecuencia de estas diferencias es que, en una composición fuerte, el contenedor es responsable del ciclo de vida de los objetos contenidos, mientras que en una composición débil, los objetos contenidos tienen un ciclo de vida independiente.

5. Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer `new` dentro de un método, o al usarlas como variables locales, ¿hablamos de composición o de "dependencia"?

Respuesta

Cuando una clase utiliza otra clase de las formas mencionadas (como parámetro, retorno, o variable local), se habla de "dependencia" y no de composición. La dependencia es una relación más débil y temporal, ya que la clase que depende de otra no mantiene una referencia permanente a ella.

Por ejemplo, si una clase `Calculadora` tiene un método que recibe un objeto `Operacion` como parámetro, la relación entre `Calculadora` y `Operacion` es de dependencia. Esto se debe a que `Calculadora` no necesita mantener una referencia a `Operacion` fuera del contexto del método. La dependencia es común en métodos utilitarios y en situaciones donde no se requiere una relación estructural duradera entre las clases.

6. En el ejemplo anterior de línea y punto, programa la relación entre `Linea` y `Punto` de dos formas. Una como composición fuerte, donde el ciclo de vida

de los puntos está ligado al de Linea y otra **como composición débil**, donde no.

Respuesta

En una composición fuerte, los puntos son creados y gestionados directamente por la línea, y su ciclo de vida está ligado al de la línea. Por ejemplo:

```
public final class LineaFuerte {
    private final Punto inicio;
    private final Punto fin;

    public LineaFuerte(double x1, double y1, double x2, double y2) {
        this.inicio = new Punto(x1, y1);
        this.fin = new Punto(x2, y2);
    }

    public double calcularLongitud() {
        return inicio.calcularDistancia(fin);
    }
}
```

En este caso, los puntos son creados dentro de la clase `LineaFuerte` y no existen fuera de ella.

En una composición débil, los puntos son creados fuera de la línea y pueden existir independientemente de ella:

```
public final class LineaDebil {
    private final Punto inicio;
    private final Punto fin;

    public LineaDebil(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public double calcularLongitud() {
        return inicio.calcularDistancia(fin);
    }
}
```

Aquí, los puntos son proporcionados a la línea desde el exterior, y su ciclo de vida no depende de la línea.

7. En Java, en la composición fuerte, ¿cuando el contenedor destruye los objetos? No se observa que `Linea` destruya los `Punto` explícitamente, ¿Por qué?

Respuesta

En Java, el contenedor (`Linea`) no destruye explícitamente los objetos (`Punto`) porque el lenguaje utiliza un sistema de recolección de basura (garbage collection). Este sistema se encarga de liberar la memoria de los objetos que ya no tienen referencias activas en el programa.

Cuando una instancia de `Linea` deja de ser utilizada y no hay más referencias a ella, el recolector de basura elimina tanto la línea como los puntos asociados, siempre que estos puntos no estén referenciados por otros objetos. Esto simplifica la gestión de memoria en comparación con lenguajes como C++, donde la destrucción de objetos debe manejarse manualmente.

8. Pon un ejemplo de composicion débil entre un departamento que tiene varios profesores. Implementa dos composiciones a la vez: entre el departamento y todos sus profesores y entre el departamento y su director, que es un profesor del departamento. Siempre debe haber un director en el departamento desde el inicio. Lanza excepciones si se viola la invariante. Emplea arrays primitivos de Java, estilo `Profesor[]` , con máximo 50, pero no rompas la encapsulación, no desveles que estás empleando un array, permite añadir un `Profesor` al final de la lista, y eliminar un profesor dada su

posición. Da acceso a los profesores con un método para saber cuántos hay y otro para obtener un profesor por posición. El director se puede cambiar por otro profesor del departamento. Sin embargo, ten en cuenta esta invariante de clase: el director debe formar siempre parte de la lista de profesores, es decir, ten cuidado al cambiar el director o al eliminar un profesor.

Respuesta

A continuación, se presenta un ejemplo que cumple con los requisitos:

```java
public class Departamento {
    private Profesor[] profesores;
    private int cantidadProfesores;
    private Profesor director;
    private static final int MAX_PROFESORES = 50;

    public Departamento(Profesor director) {
        this.profesores = new Profesor[MAX_PROFESORES];
        this.cantidadProfesores = 0;
        this.añadirProfesor(director);
        this.director = director;
    }

    public void añadirProfesor(Profesor profesor) {
        if (cantidadProfesores >= MAX_PROFESORES) {
            throw new IllegalStateException("No se pueden añadir más profesores.");
        }
        profesores[cantidadProfesores++] = profesor;
    }

    public void eliminarProfesor(int posicion) {
        if (posicion < 0 || posicion >= cantidadProfesores) {
            throw new IllegalArgumentException("Posición inválida.");
        }
        if (profesores[posicion].equals(director)) {
            throw new IllegalStateException("No se puede eliminar al director del departamento.'
        }
        for (int i = posicion; i < cantidadProfesores - 1; i++) {
            profesores[i] = profesores[i + 1];
        }
        profesores[--cantidadProfesores] = null;
    }

    public int obtenerCantidadProfesores() {
        return cantidadProfesores;
    }

    public Profesor obtenerProfesor(int posicion) {
        if (posicion < 0 || posicion >= cantidadProfesores) {
            throw new IllegalArgumentException("Posición inválida.");
        }
        return profesores[posicion];
    }
```

```java
    public void cambiarDirector(Profesor nuevoDirector) {
        boolean encontrado = false;
        for (int i = 0; i < cantidadProfesores; i++) {
            if (profesores[i].equals(nuevoDirector)) {
                encontrado = true;
                break;
            }
        }
        if (!encontrado) {
            throw new IllegalStateException("El nuevo director debe ser un profesor del departar
        }
        this.director = nuevoDirector;
    }
}

class Profesor {
    private String nombre;

    public Profesor(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}
```

Este ejemplo implementa las dos composiciones requeridas. Se asegura que el director siempre sea parte del departamento y que las operaciones de añadir, eliminar y cambiar director respeten las invariantes de la clase.